CodeMentor AI — Cómo Funciona

Análisis Estático + Compiladores Locales + Arquitectura LLM de Doble Pase (Capas K0 - K6)

TUBERÍA DE 7 CAPAS | NÚCLEO DETERMINISTA | 0% FUGA DE SOLUCIÓN | 33 ESTÁNDARES ADR

1. Diagrama de Flujo Arquitectónico



2. Principios de Funcionamiento por Capas

Capa 0: Entrada y Normalización (K0)

Se estandarizan saltos de línea (CRLF/LF) y codificación UTF-8. Se detecta automáticamente el lenguaje y entorno de motor (Unity / Unreal). Se rechazan archivos mayores a 50 KB para prevenir sobrecargas.

Capa 1: Prevalidación y Salida Temprana (K1)

Tier 0 (tree-sitter): Analiza nodos `ERROR` y `MISSING` del árbol sintáctico. Ante errores sintácticos, se emite salida temprana (~1 ms) **sin consumir llamadas LLM**.

Tier 1 (Compiladores Locales): Si gcc, clang o dotnet están disponibles, se ejecuta validación estricta de sintaxis (`-fsyntax-only`). El código **nunca se ejecuta**.

Capa 2: Análisis Estático Determinista (K2)

Consultas AST extraen hechos estructurales: profundidad de bucles, grafos de llamadas y gestión de memoria. 41 reglas escanean errores críticos. Se computa la cota Big-O de tiempo y espacio de manera determinista.

Capa 3: Ensamblado Seguro de Prompts (K3)

El código de usuario se aísla dentro de etiquetas `<user_code>` para neutralizar inyecciones de prompts. Se inyectan los hechos estructurales de K2 para evitar alucinaciones del modelo.

Capas 4 y 5: Orquestación LLM de Doble Pase y Guardia Antifugas (K4 - K5)

- **Pase A (Análisis Profundo):** Diagnostica errores lógicos y casos extremos, validado mediante esquema JSON y pruebas de línea.
- **Pase B (Mentoría Socrática):** Convierte los hallazgos en preguntas orientadoras para el estudiante.
- **Guardia Antifugas (Leakage Guard):** Filtra las preguntas producidas; si una pregunta revela la solución o el código corregido, es descartada y reemitida (0% fuga).

Capa 6: Reportes Multiformato (K6)

Unifica los resultados en formatos JSON, Markdown, el estándar industrial SARIF 2.1.0 y vistas interactivas en React Web y Escritorio Windows.

3. Sistema de Pistas Progresivas

Nivel de Pista	Alcance Pedagógico	Enfoque del Mentor
Nivel 1: Conceptual	Comprensión del concepto	"Cuando se solicita memoria en el heap, ¿cuándo termina el ciclo de vida de ese recurso?"
Nivel 2: Contexto de Línea	Líneas y variables afectadas	"Tras finalizar el bucle en la línea 8, ¿se ha liberado el puntero `tmp`?"
Nivel 3: Orientativo	Dirección de solución (sin código)	"Considere la función estándar de C para desasignar memoria antes de la sentencia return."